E4 Educator v2.0

T15

ESP-NOW

Tier 2 · Tutorial 8 of 9

Walark Electronics · Fundrum Engineering · May 2026

In this tutorial

You will learn:

- What ESP-NOW is and how it differs from WiFi and MQTT
- How to find and use MAC addresses for ESP-NOW peer registration
- How to send and receive structured data packets between ESP32 boards
- How to build a unidirectional sensor node that transmits to a master
- How to implement a star topology network with multiple sensor nodes
- The Fundrum ESP-NOW standard protocol — packet structure and CRC
- How to combine ESP-NOW with WiFi on the same device
- How to display multi-node data on the E4 TFT dashboard
- Practical range, reliability, and channel considerations

You will need:

- E4 Educator v2.0 board (master) with TFT display
- At least one additional ESP32 board to act as sensor node
- T08 completed — TFT sprite pipeline on the master
- Button.h from previous tutorials
- USB-C cable and power source for each node

1 What is ESP-NOW?

ESP-NOW is a proprietary wireless protocol developed by Espressif that allows ESP32 devices to communicate directly with each other at the radio layer, without a router, without a WiFi network, and without a broker of any kind. It uses the 2.4GHz WiFi radio but bypasses the TCP/IP stack entirely.

The result is a protocol that is extremely fast (millisecond latency), extremely power-efficient (works with deep sleep), and works in locations with no infrastructure at all — a field, a shed, a remote installation. It is ideal for sensor networks where multiple nodes need to report to one master collector.

1.1 ESP-NOW versus WiFi+MQTT

Property	WiFi + MQTT	ESP-NOW
Infrastructure	Router + broker required	No infrastructure needed
Latency	~100ms typical	~1–5ms
Range	Router range (30–100m indoor)	~200m open air, ~30–50m indoor
Max payload	Unlimited (TCP)	250 bytes per packet
Max peers	Unlimited	20 encrypted, unlimited unencrypted
Power use	High — WiFi stack always running	Very low — works with deep sleep nodes
Deep sleep compatible	Reconnection overhead on wake	Yes — send one packet and sleep immediately
Best for	Internet connectivity, dashboards, remote access	Local sensor networks, remote nodes, low power

Key point

ESP-NOW and WiFi are not mutually exclusive. The ESP32 can run both simultaneously — receiving ESP-NOW packets from sensor nodes while maintaining a WiFi connection to publish aggregated data to MQTT. The master E4 is the bridge between the ESP-NOW network and the wider internet.

1.2 MAC addresses

ESP-NOW uses MAC addresses to identify devices. Every ESP32 has a unique 6-byte MAC address burned in at manufacture. Before two devices can communicate via ESP-NOW, each must register the other as a peer using its MAC address.

```
// Print this ESP32's MAC address to Serial
#include <WiFi.h>

void setup() {
  Serial.begin(115200);
  WiFi.mode(WIFI_STA);
  Serial.printf("MAC address: %s\n",
                WiFi.macAddress().c_str());
  // Output example: MAC address: A4:CF:12:8B:2E:F1
}

void loop() {}

// Run this on each ESP32 node and note the MAC address.
```

```
// You need the MASTER's MAC to configure each sensor node.
// You need each NODE's MAC to configure the master.
```

2 Simple one-way communication

The simplest ESP-NOW use case is a sensor node sending data to a master receiver. The node has no knowledge of the master's current state — it just broadcasts readings on a schedule. This is called a unidirectional or push architecture.

2.1 The data packet structure

ESP-NOW transmits raw bytes up to 250 bytes per packet. The cleanest approach is to define a C struct for the packet, cast it to a byte array for transmission, and cast it back on the receiver. Both sender and receiver must use exactly the same struct definition.

```
// SensorPacket.h — shared between sender and receiver
// Copy this file into both projects
#pragma once
#include <Arduino.h>

// Packet identifier — helps receiver validate packet type
#define PKT_MAGIC  0xFD
#define PKT_TYPE_SENSOR 0x01

struct __attribute__((packed)) SensorPacket {
  uint8_t  magic;       // Always PKT_MAGIC (0xFD)
  uint8_t  type;        // PKT_TYPE_SENSOR
  uint8_t  nodeId;      // Unique ID for this sensor node (1-20)
  uint8_t  reserved;    // Padding for alignment
  float    tempC;       // Temperature in Celsius
  float    humidity;    // Relative humidity %
  uint16_t lightPct;    // Light level 0-100%
  uint32_t uptimeSec;   // Node uptime in seconds
  uint16_t crc;         // Simple checksum
};

// CRC: sum all bytes except last 2 (crc field itself)
uint16_t calcCRC(const uint8_t* data, size_t len) {
  uint16_t crc = 0;
  for (size_t i = 0; i < len - 2; i++) crc += data[i];
  return crc;
}

bool validateCRC(const SensorPacket& pkt) {
  return pkt.crc == calcCRC((uint8_t*)&pkt, sizeof(pkt));
}
```

2.2 Sensor node firmware (sender)

The sensor node reads its sensors, builds a SensorPacket, and sends it to the master MAC address. For battery-powered nodes, this is followed immediately by deep sleep.

```
#include <Arduino.h>
#include <WiFi.h>
#include <esp_now.h>
```

```
#include <Wire.h>
#include <Adafruit_AHTX0.h>
#include "SensorPacket.h"

// !! Replace with your master E4's MAC address !!
uint8_t masterMAC[] = {0xA4, 0xCF, 0x12, 0x8B, 0x2E, 0xF1};

#define NODE_ID  1   // Unique for each sensor node (1-20)

Adafruit_AHTX0 aht;
bool sendOk = false;

// Send callback — called after transmission attempt
void onSent(const uint8_t* mac, esp_now_send_status_t status) {
  sendOk = (status == ESP_NOW_SEND_SUCCESS);
  Serial.printf("Send %s\n", sendOk ? "OK" : "FAIL");
}

int readLDR() {
  long t = 0;
  for (int i = 0; i < 8; i++) { t += analogRead(LDR); delay(2); }
  return constrain(map(t/8, 200, 3800, 0, 100), 0, 100);
}

void setup() {
  Serial.begin(115200);
  Serial.printf("ESP-NOW Node %d starting\n", NODE_ID);

  Wire.begin(I2C_SDA, I2C_SCL);
  aht.begin(&Wire);

  // ESP-NOW requires WIFI_STA mode
  WiFi.mode(WIFI_STA);
  WiFi.disconnect();    // Not connecting — just need the radio

  if (esp_now_init() != ESP_OK) {
    Serial.println("ESP-NOW init failed.");
    return;
  }

  esp_now_register_send_cb(onSent);

  // Register master as peer
  esp_now_peer_info_t peer = {};
  memcpy(peer.peer_addr, masterMAC, 6);
  peer.channel = 0;     // 0 = use current WiFi channel
  peer.encrypt = false; // No encryption for this tutorial
  esp_now_add_peer(&peer);

  // Read sensors
  sensors_event_t h, t;
  aht.getEvent(&h, &t);
```

```
  // Build packet
  SensorPacket pkt = {};
  pkt.magic     = PKT_MAGIC;
  pkt.type      = PKT_TYPE_SENSOR;
  pkt.nodeId    = NODE_ID;
  pkt.tempC     = t.temperature;
  pkt.humidity  = h.relative_humidity;
  pkt.lightPct  = readLDR();
  pkt.uptimeSec = millis() / 1000;
  pkt.crc       = calcCRC((uint8_t*)&pkt, sizeof(pkt));

  // Send to master
  esp_now_send(masterMAC, (uint8_t*)&pkt, sizeof(pkt));

  // Wait for send callback
  delay(100);

  Serial.printf("Sent: T:%.1f H:%.1f L:%d Up:%lus\n",
                pkt.tempC, pkt.humidity,
                pkt.lightPct, pkt.uptimeSec);

  // Deep sleep for 30 seconds then repeat
  // esp_deep_sleep(30 * 1000000ULL);
  // For bench testing without deep sleep:
  delay(5000);
  ESP.restart();  // Restart to simulate wake from sleep
}

void loop() {}  // Never reached — restart() in setup()
```

WiFi.disconnect() after WiFi.mode(WIFI_STA)

ESP-NOW requires the WiFi radio in WIFI_STA mode to function. Call WiFi.mode(WIFI_STA) followed immediately by WiFi.disconnect() to ensure no association attempt is made. The radio is active but not connected to any AP. This is the correct initialisation for ESP-NOW-only nodes.

3 Master receiver

The master receives packets from all registered sensor nodes and displays their data on the TFT dashboard. It can simultaneously maintain a WiFi connection for MQTT forwarding.

3.1 Receiving packets

```
#include <Arduino.h>
#include <WiFi.h>
#include <esp_now.h>
#include <TFT_eSPI.h>
#include "SensorPacket.h"

TFT_eSPI    tft;
TFT_eSprite spr = TFT_eSprite(&tft);

#define COL_BG     0x0000
#define COL_HEADER 0x0319
#define COL_TEXT   0xFFFF
#define COL_LABEL  0xC618
#define COL_OK     0x07E0
#define COL_WARN   0xFFE0
#define COL_ERR    0xF800
#define COL_ACCENT 0xFD20
#define MAX_NODES  4

// Store latest reading from each node
struct NodeData {
  bool     valid     = false;
  float    tempC     = 0;
  float    humidity  = 0;
  uint16_t lightPct  = 0;
  uint32_t uptimeSec = 0;
  unsigned long lastSeen = 0;
  uint8_t  mac[6]    = {};
};

NodeData nodes[MAX_NODES + 1];  // Index = nodeId (1-4)
volatile bool newDataAvailable = false;
volatile uint8_t latestNodeId = 0;
SensorPacket latestPacket;

// Receive callback — called in WiFi task context
// Keep this minimal — set a flag, copy data, return
void onReceive(const uint8_t* mac, const uint8_t* data,
               int dataLen) {
  if (dataLen != sizeof(SensorPacket)) return;

  memcpy(&latestPacket, data, sizeof(SensorPacket));

  // Basic validation
  if (latestPacket.magic != PKT_MAGIC) return;
  if (!validateCRC(latestPacket)) {
```

```
    Serial.println("CRC fail — packet discarded.");
    return;
  }

  latestNodeId = latestPacket.nodeId;
  newDataAvailable = true;
}

// Process in main loop context (not callback)
void processNewPacket() {
  if (!newDataAvailable) return;
  newDataAvailable = false;

  uint8_t id = latestNodeId;
  if (id < 1 || id > MAX_NODES) return;

  nodes[id].valid     = true;
  nodes[id].tempC     = latestPacket.tempC;
  nodes[id].humidity  = latestPacket.humidity;
  nodes[id].lightPct  = latestPacket.lightPct;
  nodes[id].uptimeSec = latestPacket.uptimeSec;
  nodes[id].lastSeen  = millis();

  Serial.printf("Node %d: T:%.1f H:%.1f L:%d Up:%lus\n",
                id, nodes[id].tempC, nodes[id].humidity,
                nodes[id].lightPct, nodes[id].uptimeSec);
}

void setupESPNOW() {
  // For ESP-NOW + WiFi combined: init WiFi first
  // WiFi.mode(WIFI_AP_STA) allows both simultaneously
  // For ESP-NOW only: WiFi.mode(WIFI_STA) + disconnect
  WiFi.mode(WIFI_STA);
  WiFi.disconnect();

  if (esp_now_init() != ESP_OK) {
    Serial.println("ESP-NOW init failed."); return;
  }
  esp_now_register_recv_cb(onReceive);
  Serial.printf("ESP-NOW ready. MAC: %s\n",
                WiFi.macAddress().c_str());
}
```

Key point

The receive callback runs in a WiFi task context — not in your main loop. Never do anything slow or complex inside onReceive(). Copy the data, set a volatile flag, and return immediately. Process the data in loop() when the flag is set. This prevents stack overflow and timing issues in the callback.

4 Multi-node TFT dashboard

With multiple sensor nodes reporting to the master, the TFT dashboard shows a grid of node status cards. Each card shows the node ID, temperature, humidity, light level, and time since last contact. A node that has not been heard from for over 60 seconds is shown in a warning colour.

```
#define NODE_TIMEOUT_MS 60000   // 60 seconds

// Draw one node card at position (x, y)
void drawNodeCard(int nodeId, int x, int y) {
  int cardW = 118;
  int cardH = 118;
  NodeData& nd = nodes[nodeId];

  spr.fillSprite(COL_BG);

  if (!nd.valid) {
    // Node not yet heard from
    spr.drawRoundRect(0, 0, cardW, cardH, 8, COL_LABEL);
    spr.setTextColor(COL_LABEL, COL_BG);
    spr.setTextSize(1);
    spr.setCursor(8, 50);
    spr.printf("NODE %d", nodeId);
    spr.setCursor(8, 65);
    spr.print("waiting...");
  } else {
    unsigned long age = millis() - nd.lastSeen;
    bool stale = (age > NODE_TIMEOUT_MS);
    uint16_t borderCol = stale ? COL_ERR : COL_OK;
    uint16_t tempCol   = (nd.tempC > 28.0) ? COL_ERR :
                         (nd.tempC > 24.0) ? COL_WARN : COL_OK;

    spr.drawRoundRect(0, 0, cardW, cardH, 8, borderCol);

    // Node ID header
    spr.setTextColor(borderCol, COL_BG);
    spr.setTextSize(1);
    spr.setCursor(8, 6);
    spr.printf("NODE %d", nodeId);

    // Stale warning
    if (stale) {
      spr.setTextColor(COL_ERR, COL_BG);
      spr.setCursor(55, 6);
      spr.print("NO SIGNAL");
    } else {
      spr.setTextColor(COL_LABEL, COL_BG);
      spr.setCursor(55, 6);
      spr.printf("%lus", age / 1000);
    }

    // Temperature
    spr.setTextColor(tempCol, COL_BG);
    spr.setTextSize(2);
```

```
    spr.setCursor(8, 22);
    spr.printf("%.1fC", nd.tempC);

    // Humidity
    spr.setTextColor(COL_TEXT, COL_BG);
    spr.setTextSize(2);
    spr.setCursor(8, 46);
    spr.printf("%.1f%%", nd.humidity);

    // Light bar
    spr.setTextColor(COL_LABEL, COL_BG);
    spr.setTextSize(1);
    spr.setCursor(8, 74);
    spr.printf("Light %d%%", nd.lightPct);
    spr.drawRect(8, 86, 102, 8, COL_LABEL);
    int fw = map(nd.lightPct, 0, 100, 0, 100);
    if (fw > 0) spr.fillRect(9, 87, fw, 6, COL_ACCENT);

    // Uptime
    spr.setTextColor(COL_LABEL, COL_BG);
    spr.setCursor(8, 100);
    spr.printf("Up: %lus", nd.uptimeSec);
  }

  spr.pushSprite(x, y);
}

// Draw all four node cards in a 2x2 grid
void updateNodeGrid() {
  // Card positions in 240x280 content area
  int positions[4][2] = {
    {1,   42},   // Node 1: top-left
    {121, 42},   // Node 2: top-right
    {1,   162},  // Node 3: bottom-left
    {121, 162},  // Node 4: bottom-right
  };

  spr.createSprite(118, 118);  // Resize for card
  for (int i = 1; i <= MAX_NODES; i++) {
    drawNodeCard(i, positions[i-1][0], positions[i-1][1]);
  }
  spr.deleteSprite();
}

// In setup():
  tft.fillScreen(COL_BG);
  tft.fillRect(0, 0, 240, 40, COL_HEADER);
  tft.setTextColor(COL_TEXT, COL_HEADER);
  tft.setTextSize(2);
  tft.setCursor(8, 12);
  tft.print("ESP-NOW Network");

// In loop():
  processNewPacket();
```

```
// Refresh grid every second to update age timers
static unsigned long lastGrid = 0;
if (millis() - lastGrid >= 1000) {
  lastGrid = millis();
  updateNodeGrid();
}
```

5 ESP-NOW and WiFi simultaneously

The master E4 can receive ESP-NOW packets from sensor nodes and simultaneously maintain a WiFi connection for MQTT forwarding. This requires using WIFI_AP_STA mode and matching the WiFi channel to the ESP-NOW channel.

5.1 The channel problem

ESP-NOW operates on a specific 2.4GHz channel (1-14). When a WiFi connection is established, the ESP32 moves to the AP's channel. ESP-NOW communication can only happen on the current active WiFi channel. This means sensor nodes must transmit on the same channel as the master's WiFi connection.

The solution is to set the channel on the master after WiFi connects, and configure all sensor nodes to use that channel explicitly:

```
// On the master — after WiFi.begin() connects:
int currentChannel = WiFi.channel();
Serial.printf("WiFi channel: %d\n", currentChannel);
// Tell sensor nodes to use this channel (out of band —
// hard-code in node firmware or store in NVS)

// Master ESP-NOW init with WiFi active:
void setupESPNOWWithWiFi() {
  // Connect WiFi first
  WiFi.mode(WIFI_AP_STA);   // Must be AP_STA not STA alone
  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED) delay(100);

  int ch = WiFi.channel();
  Serial.printf("WiFi connected on channel %d\n", ch);

  // Init ESP-NOW after WiFi connected
  esp_now_init();
  esp_now_register_recv_cb(onReceive);
}

// On sensor nodes — use the master's WiFi channel:
#define ESPNOW_CHANNEL  6   // Match master WiFi channel

esp_now_peer_info_t peer = {};
memcpy(peer.peer_addr, masterMAC, 6);
peer.channel = ESPNOW_CHANNEL;  // Explicit channel
peer.encrypt = false;
esp_now_add_peer(&peer);

// Also set the node's wifi channel before sending:
esp_wifi_set_channel(ESPNOW_CHANNEL, WIFI_SECOND_CHAN_NONE);
```

Key point

The WiFi channel issue is the most common ESP-NOW problem when combining with WiFi. If your nodes can communicate without WiFi but fail when WiFi is active, the channel mismatch is almost certainly the cause. Use WiFi.channel() on the master after connection and hard-code that value into your node firmware.

5.2 MQTT bridge — forwarding node data

Once the master receives a packet from a node, it can forward the data to MQTT. Node-RED then receives readings from all nodes via a single MQTT connection:

```
// In processNewPacket() — after storing to nodes[] array:
void forwardToMQTT(uint8_t nodeId) {
  if (!mqtt.connected()) return;

  NodeData& nd = nodes[nodeId];
  char topic[32];
  char payload[16];

  // e4/node/1/temperature, e4/node/2/temperature etc.
  snprintf(topic, sizeof(topic), "e4/node/%d/temperature", nodeId);
  snprintf(payload, sizeof(payload), "%.1f", nd.tempC);
  mqtt.publish(topic, payload, true);

  snprintf(topic, sizeof(topic), "e4/node/%d/humidity", nodeId);
  snprintf(payload, sizeof(payload), "%.1f", nd.humidity);
  mqtt.publish(topic, payload, true);

  snprintf(topic, sizeof(topic), "e4/node/%d/light", nodeId);
  snprintf(payload, sizeof(payload), "%d", nd.lightPct);
  mqtt.publish(topic, payload, true);

  Serial.printf("MQTT: forwarded node %d\n", nodeId);
}
```

6 Fundrum ESP-NOW standard protocol

For more complex multi-project ESP-NOW networks, Fundrum Engineering maintains a standard packet protocol that provides a consistent header, message type system, and CRC across all ESP-NOW projects. This is the protocol used in the THETYS and DRONELINE family of devices.

6.1 Protocol header structure

Byte(s)	Field	Value	Description
0	magic	0xFD	Fundrum magic byte — identifies packet as Fundrum protocol
1	version	0x10	Protocol version 1.0
2	family	0x01–0xFF	Device family identifier (high nibble = type, low nibble = subtype)
3	msgType	0x01–0xFF	Message type within the family
4	nodeId	0x01–0x14	Sending node ID (1-20)
5	seqNum	0x00–0xFF	Sequence number — wraps at 255. Detects missed packets.
6-7	payloadLen	0–242	Length of payload following the 8-byte header
6+len-1	crc8	—	CRC-8/Dallas-Maxim over all bytes except crc8 itself

The payload (bytes 8 onwards) is a packed struct specific to the message type. For a sensor reading message, it contains temperature, humidity, battery voltage, and any other sensor values relevant to that node family.

6.2 Sequence numbers and missed packet detection

The seqNum field increments on each transmission. The master tracks the last received seqNum per node. If the received seqNum is not lastSeqNum+1, one or more packets were missed:

```
uint8_t lastSeqNum[MAX_NODES + 1] = {};

void checkSequence(uint8_t nodeId, uint8_t seqNum) {
  uint8_t expected = lastSeqNum[nodeId] + 1;
  if (seqNum != expected && lastSeqNum[nodeId] != 0) {
    uint8_t missed = seqNum - expected;
    Serial.printf("Node %d: %d packet(s) missed\n",
                  nodeId, missed);
  }
  lastSeqNum[nodeId] = seqNum;
}
```

7 Practical considerations

7.1 Range and reliability

Condition	Typical range
Open air, line of sight	150–200m with stock PCB antenna
Indoor, same room	20–50m reliably
Indoor, through walls	10–30m depending on wall construction
Metal enclosure	Severely reduced — route antenna outside enclosure
With external antenna	300–500m+ open air (requires WROVER or external antenna module)

7.2 Common ESP-NOW problems

Symptom	Cause	Solution
esp_now_init() returns error	WiFi mode not set first	Call WiFi.mode(WIFI_STA) before esp_now_init()
Callback never fires	Peer MAC wrong or not registered	Verify MAC with the print-MAC sketch. Check esp_now_add_peer() returns ESP_OK.
Works without WiFi, fails with	Channel mismatch	Set node channel to match master WiFi channel. Use WIFI_AP_STA on master.
Packet received but data corrupt	Struct size mismatch or padding	Use __attribute__((packed)) on struct. Verify sizeof() matches on both devices.
CRC fails intermittently	Radio interference	Normal at range limits. Add retry logic in sender. Log missed packets with seqNum.
Sends succeed but receiver misses	Callback too slow or blocking	Keep onReceive() minimal. Copy + flag only. Process in loop().

8 T15 summary

Concept	Key takeaway
ESP-NOW	Direct ESP32–ESP32 wireless, no router, no broker. 250 bytes max, ~200m range, millisecond latency.
MAC addresses	Unique 6-byte hardware address. Use WiFi.macAddress() to print. Both sender and receiver must register each other as peers.
WiFi.mode(WIFI_STA)	Required before esp_now_init(). Follow with WiFi.disconnect() on node-only devices.
Packet struct	Use __attribute__((packed)) structs shared between sender and receiver. Verify sizeof() matches. Include magic byte and CRC.
onReceive() callback	Runs in WiFi task context. Keep minimal — copy data and set volatile flag only. Process in loop().
ESP-NOW + WiFi	Use WIFI_AP_STA mode on master. Set node channel to match master WiFi channel. Init ESP-NOW after WiFi connects.

Node timeout	Track lastSeen timestamp per node. Show warning colour after NODE_TIMEOUT_MS (60s). Detects dead or out-of-range nodes.
Sequence numbers	Increment per packet. Compare received vs expected on master. Detects missed packets without acknowledgement overhead.
MQTT bridge	Master forwards ESP-NOW data to MQTT topics e4/node/N/measurement. Node-RED sees all nodes via one MQTT connection.

What's next

- T16 — I2S audio and MEMS microphone: the final Tier 2 tutorial covers the ICS43434 MEMS microphone already on the E4, the I2S peripheral, real-time FFT analysis, and displaying a live audio spectrum on the TFT. This is the most technically challenging tutorial in Tier 2 and the natural gateway to the Tier 3 BANDSCOPE project.